

Typed Non-Laundering Authorization for Signed Deterministic-Program Envelopes

Anonymous Author(s)

Abstract

We study authenticated recovery of signed deterministic programs over channels that do not preserve bytes. We separate signed programs from signed execution envelopes and define a typed, machine-checked non-laundering model over verifier-aggregated report sets. Policy can grant reasoned origin-only use, but cannot rename evidence or derive execution-agreement authority from missing, incomparable, or disagreeing execution. Independence policy receives only an eight-bit verified projection; a Distinct axis qualifies only with registry-verified, externally attested, or verifier-established provenance and its required evidence reference.

On a bounded surface of the real deterministic DSL L , a **160/160** oracle separated program-only A0, authentication-only A1, final-result re-execution B, and transcript re-execution C. Observation-divergent stale-3 is unconstructible at A0, missed at A1, and detected throughout B/C. Four programs through five paths produced byte-identical canonical transcripts in fresh-process reruns; a distinct cross-language backend detected one controlled Rust-runtime common-mode fault missed by shared-host paths. The result demonstrates bounded backend separation, not source-independent replication.

CCS Concepts

• Security and privacy → Software and application security.

Keywords

authorization, attestation, deterministic re-execution, provenance envelopes, non-laundering

1 Introduction

Authenticated origin must be separated from decision authority. For signed deterministic programs, a recipient must distinguish the exact program and context, an attributed execution claim, local deterministic agreement with that claim, and authority for an action. A signature authenticates the claim rather than proving that the computation occurred or was correct. Re-execution supplies only case-specific agreement, and shared faults or unsupported profiles remain possible. Treating these states as interchangeable launders authority.

DSSE, in-toto, SLSA, and Sigstore establish exact-message or provenance facts [6, 9, 10, 12], but not verifier-side execution agreement or the later authorization decision. We encode that downstream non-laundering boundary in a typed state model and test the incremental detection value of re-execution on real engines.

SignedProgramV1 binds canonical L IR, semantics, and context. SignedExecutionEnvelopeV1 adds a versioned engine-attributed claim. The verifier authenticates exact bytes under external trust policy, checks context, and conditionally re-executes L . VerificationStatus records terminal execution evidence and Author

izationBasis a later grant without rewriting it. Authorization consumes the complete current-report aggregate for one envelope, context, scope, and epoch. Disagreement dominates fallback, while non-execution and incomparability retain typed origin-only reasons. The structural theorem covers any predicate over the verified Boolean projection. The executable model checks 1,024 conjunctive-monotone policies and three laundering mutations.

The real-engine matrix spans four families and five paths on a bounded L surface. I3 compares the native reference with a cross-language transliteration that shares source, parser, and semantic-core lineage but separates runtime backend and build toolchain. Across 160 cells, observation-divergent stale-3 is N/A at A0, missed at A1, and detected by B/C. A controlled Rust-host common-mode fault is missed at I1/I2 and detected by E4 at I3. This tests backend, not source-lineage, separation. The evaluation states all coverage and independence limits.

1.1 Contributions

This paper makes two contributions in claim order:

- (1) **A typed, machine-checked non-laundering discipline.** Split signed types, complete-set joins, typed fallback reasons, and an eight-axis verified projection prevent policy from relabeling evidence. The structural result covers arbitrary projected predicates. The finite model checks 9,437,184 state-policy pairs, 23,639,903 ordered report triples, and 31,744 forbidden-origin path families and catches three mutations.
- (2) **A reproducible real-engine detection result.** A bounded 160-cell matrix passes **160/160** oracle checks, and four programs across five paths yield byte-identical canonical transcripts on rerun. A0/A1/B/C isolate claim existence, binding, and re-execution. B/C detect every observation-divergent stale-3 injection and controlled faults according to the recorded profiles. E4 remains a transliteration, so no source-independent replication is claimed.

2 System Model and Verification Pipeline

2.1 Roles and data flow

The publisher signs a canonical SignedProgramV1 for A0 or adds profiles and an execution claim in SignedExecutionEnvelopeV1 for A1/B/C. A1 disables re-execution after context checking and records not_reexecuted(skipped_by_policy). B/C perform verifier re-execution under final-result and transcript commitments. After channel recovery, the verifier parses one canonical variant, authenticates it under external trust policy, checks bootstrap hints against signed values and signed context, conditionally re-executes L , aggregates every current report, and only then applies promotion policy.

The decomposition is:

```

117 Recover(y) -> m | bottom carrier layer
118 Authenticate(m, TrustPolicy) origin layer
119 CheckContext(e, ContextPolicy) context/freshness layer
120 CheckExecution(e, EnginePolicy) execution-agreement layer
121 Authorize(Aggregate(envelope_id, context_id,
122     verifier_scope, all_current_reports),
123     PromotionPolicy) authorization layer

```

No layer inherits the authority of the next. Recovery or decoder confidence is not authentication. Authentication is not execution agreement. Execution agreement is not authorization. Any outer carrier label and embedded key identifier are bootstrap hints only; neither can create a trust root.

2.2 Why determinism is load-bearing

Let $\mathbb{B}^*$ be finite byte strings, $I \subseteq \mathbb{B}^*$ canonical L intermediate representations, S semantics profiles, G execution profiles, and $\mathcal{T}$ finite execution transcripts. For $s \in S$ and $g \in G$, the reference semantics is a partial function

$$\text{Exec}_{s,g} : I \rightarrow \mathcal{T}.$$

Partiality represents unsupported profiles, exhausted local limits, or other conditions under which the verifier cannot make the signed claim comparable. It does not introduce nondeterministic outcomes.

Assumption D (determinism of L). For all $i \in I$, $s \in S$, and $g \in G$, if $\text{Exec}_{s,g}(i)$ is defined, then it has a unique value. Equivalently, two conforming evaluations under the same canonical IR, semantics profile, execution profile, and modeled inputs produce the same finite transcript.

Let q be a versioned commitment scope and $\text{Project}_q : \mathcal{T} \rightarrow \mathbb{B}^*$ its deterministic canonical projection. A scope names its observation axes—for example, selected values, standard output, diagnostics, termination status, warning order, or resource reports—and makes no statement about axes outside that set. For transcript τ , define

$$\begin{aligned}
 \text{td}_q(\tau) &:= H(\text{Project}_q(\tau)), \\
 \text{od}(\tau) &:= \text{Outcome}(\tau), \\
 \text{dd}(\tau) &:= H(\text{DetEnc}(\text{Diagnostics}(\tau))).
 \end{aligned}$$

Execution agreement holds only when local profiles and transcript schema are comparable, local execution is defined, and these recomputed values equal the signed transcript_digest, claimed_outcome, and diagnostics_digest. Agreement is therefore a case-specific equality test against a signed claim. Determinism makes it reproducible. It does not establish that the attributed publisher-side engine ran, when it ran, or that either implementation is defect-free.

3 Authoritative Signed Program and Execution Types

3.1 Typed envelope

All strings and tagged unions have one deterministic byte encoding. Digests carry algorithm identifiers, and profile identifiers resolve only through verifier-approved registries or local policy.

```

171 CanonicalProgramIR {
172   source: Bytes;
173   normalized_ir: Bytes;

```

```

175 }
176
177 EngineDescriptor {
178   implementation_name: Identifier;
179   source_revision: Identifier;
180   executable_or_module_digest: Digest;
181   build_profile_digest: Digest;
182   dependency_lock_digest: Digest;
183   runtime_target: Identifier;
184   semantics_profile_digest: Digest;
185 }
186
187 ExecutionClaim {
188   transcript_schema: Identifier;
189   commitment_scope: CommitmentScope;
190   transcript_digest: Digest;
191   claimed_outcome: Outcome;
192   diagnostics_digest: Digest;
193   execution_profile: ExecutionProfile;
194   attributed_engine: EngineDescriptor;
195 }
196
197 EnvelopeContext {
198   purpose: Identifier;
199   subject: Identifier;
200   issuer: Identifier;
201   nonce: Nonce;
202   issued_at: Timestamp;
203   expires_at: Timestamp;
204 }
205
206 SignedProgramV1 {
207   canonical_ir: CanonicalProgramIR;
208   language_profile: Identifier;
209   semantics_digest: Digest;
210   context: EnvelopeContext;
211 }
212
213 SignedExecutionEnvelopeV1 {
214   program: SignedProgramV1;
215   execution_claim: ExecutionClaim;
216 }
217
218 SignedObjectV1 {
219   domain: Identifier;
220   object_version: Version;
221   codec_profile: Identifier;
222   payload_tag: Program | Execution;
223   payload: SignedProgramV1 | SignedExecutionEnvelopeV1;
224 }

```

Scope A0's payload is exactly SignedProgramV1. Scopes A1, B, and C use SignedExecutionEnvelopeV1 with a nonempty versioned commitment scope; A1 and B use the same final-result claim shape, while only B enables verifier re-execution. payload_tag must match the selected payload type. The source is not a missing top-level field: it is the source member of canonical_ir, beside the normalized IR bytes, and both are covered by the signed deterministic encoding. Well-formedness for either payload requires the fixed V1 framing, an allowed L language profile, canonical L IR, internally consistent semantics digests, and ordered context timestamps. Only the execution-bearing type requires an execution claim and nonempty commitment scope. External policy may be stricter.

Let DetEnc be injective and deterministic over these typed values and PAE injectively domain-separate the message. For payload e ,

$$\begin{aligned}\mu(e) &:= \text{PAE}(\text{DOMAIN}_{V1}, \text{DetEnc}(e)), \\ \sigma &:= \text{Sign}(sk, \mu(e)), \\ m &:= \text{DetEnc}(e, \sigma).\end{aligned}$$

Here e is the complete `SignedObjectV1` and DOMAIN_{V1} is a fixed public domain-separation string whose concrete prototype spelling is blinded. Every listed framing and payload field is under one signature. Signing only `canonical_ir`, or signing separately replaceable blocks, is nonconforming because it reopens version mixing, profile substitution, stale-result splicing, and status laundering.

The construction is signature-scheme-agnostic: an approved scheme must provide EUF-CMA security under the verifier's trust policy. Ed25519 is the intended deployment choice. The dependency-free RQ3 mechanism harness instead used HMAC-SHA256 as a stand-in for producing and checking a single authenticator over the same domain-separated bytes. No security claim or comparative result rests on HMAC behaving as a public-key signature or on the choice of a particular primitive.

3.2 Meaning of the execution claim

An **engine-attributed execution claim** is a publisher-signed statement that a particular observation tuple is attributed to the implementation described by `attributed_engine`. **Verified execution agreement** is the verifier's finding that its deterministic re-execution yields the same committed observation. An **execution proof** would verify computation correctness without re-execution. This construction supplies the first notion and can derive the second; it supplies no execution proof.

The signature makes the attribution immutable as part of e . It does not attest the occurrence of publisher-side execution. Operationally, the commitment is a promotion contract: execution-agreement authority is unavailable unless a comparable verifier-side re-execution agrees with the signed observation. The comparison can expose version drift, a newly signed stale claim, or implementation divergence, while remaining agnostic about whether the publisher obtained the observation by execution, prediction, or another process.

`attributed_engine` has exactly two meanings: the signer attributes the signed execution claim to the identified implementation, and the descriptor supplies publisher-asserted inputs for constructing an `IndependenceProfile`. Its signature proves only that those inputs are unchanged. It does not prove their independence assertions, attest that the described binary existed at a place or time, or show that it was loaded, executed this IR, produced the claim, had a trustworthy build, or returned a correct result. Section 4.1 therefore requires independent provenance before an axis can support high-authority promotion.

3.3 Stale claims and bound fields

For `SignedExecutionEnvelopeV1`, the whole-object signature distinguishes three stale cases:

- (1) **stale-1:** attaching a result digest from a different envelope to the current IR, or otherwise mixing an IR and result from different envelopes, changes signed bytes and fails authentication;

- (2) **stale-2:** replaying an old but valid whole envelope does not forge a signature and is addressed, when required, by signed purpose, subject, nonce, expiry, and freshness policy; and
- (3) **stale-3:** signing an old result as a new claim for the current IR can pass authentication and freshness. Re-execution detects such a freshly signed stale claim when its committed observation differs from the current deterministic execution. A stale claim that remains observation-equal under its commitment scope is not detected by this comparison.

Full intermediate-trace differences are detectable only when the versioned `commitment_scope` includes those observation axes. A final-result commitment makes no completeness claim about uncommitted intermediate behavior.

4 Typed Verification and Authorization

4.1 Independence is a vector

An independence profile records relationships between the publisher-attributed and verifier-side execution paths:

```
Relation := Shared | Partial | Distinct | Unknown
Provenance := PublisherAsserted | RegistryVerified |
  ExternallyAttested | VerifierEstablished

AxisEvidence {
  relation: Relation;
  provenance: Provenance;
  evidence_ref: Digest if provenance != PublisherAsserted
    else Optional<Digest>;
}

IndependenceProfile {
  source_lineage: AxisEvidence;
  parser_lineage: AxisEvidence;
  semantics_core_lineage: AxisEvidence;
  build_toolchain: AxisEvidence;
  runtime_backend: AxisEvidence;
  dependency_set: AxisEvidence;
  hardware_architecture: AxisEvidence;
  operator_or_organization: AxisEvidence;
}
```

`RegistryVerified` requires a registry-entry digest, `ExternallyAttested` an attestation digest, and `VerifierEstablished` a local evidence-record or derivation identifier. Only `PublisherAsserted` may omit `evidence_ref`. Provenance validation remains external to `AxisEvidence`; a missing or invalid required reference makes the axis ill-formed before projection, and `PublisherAsserted` never satisfies high-authority independence.

The high-authority independence predicate for one axis is

$$\text{Independent}(a) \iff a.\text{relation} = \text{Distinct} \wedge a.\text{provenance} \in \text{VerifiedProvenance},$$

where `VerifiedProvenance` is exactly `{RegistryVerified, ExternallyAttested, VerifierEstablished}` and well-formedness already requires the corresponding `evidence_ref`. `PublisherAsserted` is insufficient even when the signed relation says `Distinct`; the signature authenticates the assertion but does not make it true. `Partial`, `Unknown`, and `Shared` also project to false. We do not assume an honest publisher for independence metadata.

Policy cannot inspect raw AxisEvidence. Use the eight-axis declaration order above and define

$$\mathcal{I}(P) := (\text{Independent}(P_1), \dots, \text{Independent}(P_8)) \in \{0, 1\}^8, \\ \phi : \{0, 1\}^8 \rightarrow \{0, 1\}.$$

The execution-agreement guard is exactly $\phi(J)$, where J is the aggregate projection defined below; policy never receives P . Thus PublisherAsserted + Distinct becomes zero before policy sees it and cannot be granted by a raw-relation predicate. A conforming promotion policy is any total predicate over this verified Boolean projection that also obeys the status-preserving transitions and the pinned verifier scope below; it may be disjunctive or non-monotone over the eight bits. Labels I0–I3 may summarize profiles for exposition, but they do not form a total order. Agreement raises differential detection sensitivity only for fault domains actually separated by the concrete supporting profiles. It does not exclude shared defects, coincident defects, common specification ambiguity, or a fault inactive on the tested input.

4.2 Separate evidence from authority

Let the report-level evidence-derived status be

$$\text{VS} := \{\text{execution_agreed}(J), \text{execution_disagreed}, \\ \text{not_reexecuted}(R_{nr}), \text{not_comparable}(R_{nc})\}.$$

Here $J \in \{0, 1\}^8$. An agreement report carries only $J_i = \mathcal{I}(P_i)$; its concrete P_i and axis references live in the supporting-evidence record committed by supporting_evidence_digest. Thus report-level VerificationStatus has exactly four variants; authenticated is a pipeline state. R_{nr} is a nonempty set over skipped_by_policy, engine_unavailable, unsupported_profile, and resource_budget_declined. R_{nc} is a nonempty set of typed comparison failures such as profile mismatch, unsupported scope, schema mismatch, or undefined execution. Every report is keyed by (envelope_id, context_id, verifier_scope), has an immutable report identifier and creation epoch, optional supersession epoch, supporting-evidence digest, and exactly one status. For a key and epoch, current means the complete set of matching reports created by then and not superseded at or before it; caller-supplied subsets are not current sets.

Authorization never consumes one caller-selected report. The verifier computes

```
Aggregate(envelope_id, context_id, verifier_scope,
          aggregation_epoch, verifier_store)
```

The verifier, not the consumer, selects from its owned store every matching report created no later than the epoch and not superseded at or before it. This internally derived complete current set includes older unsuperseded reports; the aggregation API accepts no caller-supplied report iterable. Write D for disagreement, $A_I(J) := \text{execution_agreed}(J)$ for agreement, $C(R)$ for incomparability, and $N(R)$ for non-execution. Each agreement report exposes $A_I(J_i)$. Aggregate status folds one binary join $\sqcup$ with identity $\perp$; when agreement wins, $J = \bigcap_i J_i$. The precedence is

$$D > A_I(J) > C(R) > N(R) > \perp.$$

For unequal kinds, the higher kind wins. For equal kinds,

$$A_I(J) \sqcup A_I(K) = A_I(J \cap K), \\ C(R) \sqcup C(S) = C(R \cup S), \\ N(R) \sqcup N(S) = N(R \cup S).$$

Disagreement is absorbing. This single algebra has identity, associativity, commutativity, and idempotence; multiple agreements intersect high-authority axes without constructing new provenance evidence, while like-kind fallback reports union reason sets. Each underlying verifier-store report commits its concrete profile and evidence references through supporting_evidence_digest. A later disagreement therefore supersedes an earlier non-execution origin-only authorization regardless of report order. The consumer receives only the verifier's aggregate and cannot cherry-pick a member report.

Let

$$\text{AB} := \{\text{origin_only}(r), \text{execution_agreement}\},$$

where the typed fallback reason r contains the complete not_reexecuted or not_comparable reason set that authorized origin-only use. An authorization state is $(s, b) \in \text{VS} \times \text{Option}(\text{AB})$. None means no authorization; it is not a third authorization basis. For promotion policy Π_P , the grant transitions from the verifier-computed aggregate are

$$\begin{aligned} &(\text{not_reexecuted}(R_{nr}), \text{None}) \rightarrow_{\Pi_P} \\ &(\text{not_reexecuted}(R_{nr}), \text{origin_only}(R_{nr})) \\ &\text{if origin-only use is explicitly allowed,} \\ &(\text{not_comparable}(R_{nc}), \text{None}) \rightarrow_{\Pi_P} \\ &(\text{not_comparable}(R_{nc}), \text{origin_only}(R_{nc})) \\ &\text{if origin-only use is explicitly allowed,} \\ &(A_I(J), \text{None}) \rightarrow_{\Pi_P} \\ &(A_I(J), \text{execution_agreement}) \\ &\text{if execution use is allowed and } \phi_{\Pi_P}(J). \end{aligned}$$

not_comparable(reason) $\rightarrow$ origin_only(reason) does not relabel incomparability as agreement. This removes only one of two skip incentives: attempting and failing into not_comparable no longer loses to skipping, but a party that anticipates disagreement can still skip to obtain origin_only rather than attempt and receive execution_disagreed; origin_only is the honest weaker floor, never execution_agreement, and a later disagreement supersedes it if re-execution occurs. Disagreement has no grant, and neither fallback status has an execution-agreement grant. Every authorization output records its exact basis and typed reason plus verifier_scope_digest, aggregation_epoch, report_set_digest, and promotion_policy_digest. report_set_digest uses a canonical typed encoding of each current report's globally unique identifier, status, creation epoch, and supporting-evidence digest, not merely the final join. Current-set membership is evaluated at aggregation_epoch: a report exists by that epoch and has not been superseded at or before it, while later reports and future supersessions cannot alter the historical snapshot. The action policy must pin an allowed verifier_scope_digest; a mismatched scope yields no grant, so a consumer cannot request a more favorable scope. Execution

agreement records the intersected verified projection; its concrete supporting profiles and axis evidence remain committed through the report-set digest.

4.3 Non-laundering invariant

The first invariant says policy may attach authority but cannot rewrite evidence:

$$q \rightarrow_{\Pi_P} q' \implies \text{status}(q') = \text{status}(q). \quad (\text{NL-1})$$

Let B contain `execution_disagreed`, every `not_reexecuted(reason)`, and every `not_comparable(reason)`. The central safety property is

$$\begin{aligned} s \in B \implies \neg \exists q'. (s, \text{None}) \rightarrow_{\Pi_P}^* q' \\ \wedge \text{basis}(q') = \text{execution_agreement}. \end{aligned} \quad (\text{NL-2})$$

Policy may decide that already-established origin evidence is sufficient for a named low-risk use and must record why. It may not relabel that evidence or report that execution agreement occurred.

4.4 Safe promotion and finite-model safety check

Theorem 1 (execution-agreement non-laundering). For every adversary and every conforming promotion policy, verifier-side aggregation is independent of report order, a policy transition preserves the aggregate evidence-derived `VerificationStatus`, and no aggregate with status `execution_disagreed`, `not_reexecuted(reason)`, or `not_comparable(reason)` can reach an authorization whose basis is `execution_agreement`. A `Distinct` axis contributes to high-authority profile acceptance only with registry-verified, externally attested, or verifier-established provenance and its required evidence reference.

Argument. Aggregate folds the associative and commutative join over the complete current report set, with disagreement dominant; thus adding disagreement replaces any prior non-execution aggregate regardless of report order. The only rule emitting basis `execution_agreement` has source $A_I(J)$ and guard $\phi_{\Pi_P}(J)$. Every grant rule preserves the complete projected status or typed fallback reason; the report-set digest separately preserves the supporting-evidence commitments. The three prohibited source classes have no transition to execution-agreement authority. NL-1 follows inductively over finite policy paths; NL-2 follows by reachability induction. Required evidence references are checked before $\mathcal{I}$, and policy never receives raw P_i , so a valid signer cannot create high-authority independence merely by asserting `Distinct`. Scope pinning independently prevents selection of a different verifier aggregate. Under Assumption D, every reachable execution-agreement authorization is backed by a reproducible local execution whose committed observation equals the signed claim under the recorded supporting profiles.

This argument does not require ϕ to be monotone, conjunctive, or compositional over the eight projected bits. For any predicate sets— $\phi(J)$ is only a guard on the edge from an already established aggregate $A_I(J)$. Changing that guard can add or remove

authorizations among agreed projections, but it cannot create an execution-agreement edge from a prohibited status or change the source status. Non-laundering is therefore a structural property of the transition relation, independent of the projected predicate's shape.

Mechanical safety evidence. `non_laundering_model.py` enumerates all $4^8 = 65,536$ concrete quaternary relation vectors and all 32 one-axis `Relation` $\times$ `Provenance` $\times$ `evidence_ref-presence` cases. The projection maps an axis to one exactly when relation is `Distinct`, provenance is `RegistryVerified`, `ExternallyAttested`, or `VerifierEstablished`, and the required reference exists. Conjunctive policy receives only the resulting bit vector, so the transition checker exhausts its exact 2^8 authorization-equivalence quotient and $2 \cdot 2 \cdot 2^8 = 1,024$ policies. With typed fallback reasons and bases, it checks 9,216 raw states, 9,437,184 state-policy pairs, 28,482 generated policy edges, 322,370 reachable edges, and 31,744 forbidden-origin path families. The join checks cover 287 singleton statuses, 82,369 ordered status pairs, and all 23,639,903 ordered status triples, including 30 non-execution/disagreement supersession orders. Another 1,024 cases reject an unpinning verifier scope and verify all four authorization digest/epoch outputs; seven verifier-store cases check older-current inclusion, future-report exclusion, before/after-epoch supersession, canonical status and supporting-evidence digest separation and historical stability, and globally unique report identifiers. “Exhaustive” remains scoped to this finite conjunctive-monotone policy language, not arbitrary projected predicates ϕ .

The first mutation rewrites `not_reexecuted(reason)` to `execution_agreed` and grants that basis. The second corrupts the Boolean projector so `Distinct` + `PublisherAsserted` becomes one. The third passes raw P to a predicate that grants on that same publisher assertion. The checker rejects the first at status immutability and both provenance mutations against the verified-projection authorization invariant. The executable abstraction covers join laws, disagreement dominance, current-set metadata, scope pinning, authorization digests, status immutability, basis provenance, typed fallback reasons, and typed terminal failure. It does not encode stage monotonicity or signed-value authority and is not a proof that an arbitrary implementation conforms to the model.

5 Staged Verification Semantics

For fixed external policies $\Pi = (\Pi_T, \Pi_C, \Pi_E, \Pi_P, \Pi_F)$, where Π_F includes allowed formats and profiles, local resource limits, and freshness requirements, define:

```
PipelineState := ...
    | authenticated(...) | context_checked(...)
    | verified(e, r, s) | aggregated(e, c, v, R, s)
    | authorized(e, c, v, R, s, b)
    | nonpromoted(e, c, v, R, s) | reject( $\rho$ ).
```

Here $r \in \{\text{exact}, \text{robust}\}$, c is the context identifier, v the verifier scope, R the complete current report set, $s \in \text{VS}$, $b \in \text{AB}$, and ρ is a typed rejection reason. `authenticated(...)` and `context_`

checked(...) are pipeline variants, never report-level statuses. The least transition relation follows nine stages:

- (1) **Bootstrap.** A fixed, non-extensible grammar parses an untrusted candidate. A decoder profile is selected only from the external allowlist. Unknown profiles or failure reject.
- (2) **Recovery.** The decoder returns candidate bytes or failure. The report records exact if exactRecover holds and otherwise robust if robustRecover holds. Confidence alone creates no successor.
- (3) **Structural validation.** The candidate must parse as exactly one canonical (e, σ) , satisfy V1 types and bounds, and re-encode byte-for-byte to m .
- (4) **Authentication.** Candidate keys come only from external trust policy with issuer and purpose constraints. An embedded key cannot add itself to that policy.
- (5) **Bootstrap recheck.** Every security-relevant outer value used for decoding is compared with its authenticated internal counterpart. A mismatch rejects; an outer label never overrides a signed value.
- (6) **Context.** Signed purpose, subject, issuer, nonce, and timestamps are checked. Success gives a context-checked object, not authorization.
- (7) **Execution classification.** Exactly one report-level status is recorded: not_reexecuted(reason); not_comparable(reason) when comparison cannot complete; execution_disagreed; or execution_agreed(J_i), where $J_i = I(P_i)$ and supporting_evidence_digest commits P_i and its evidence references. A later execution adds a new immutable report rather than mutating the prior report.
- (8) **Aggregation.** The verifier, not the consumer, joins all current reports for the envelope, context, and verifier scope. Disagreement dominates and invalidates any earlier origin-only fallback in that context; multiple agreement reports yield $A_I(J)$ with $J = \bigcap_i J_i$.
- (9) **Authorization.** Policy consumes only the aggregate. Origin-only grants preserve their typed fallback reason; execution authority requires an already established $A_I(J)$ satisfying $\phi_{\Pi_P}(J)$. All other results are typed nonpromotion.

Every reachable configuration satisfies stage monotonicity, signed-value authority, complete-set aggregation, status immutability under policy, basis provenance, and typed terminal failure. Earlier-stage success never implies later-stage authority. non_laundering_model.py covers aggregate order and disagreement dominance, status immutability, basis provenance, provenance-gated independence, typed fallback reasons, and typed terminal failure; it does not model stage monotonicity or signed-value authority.

6 Threat Model and Security Games

6.1 Adversary and assumptions

The adversary may submit, edit, transform, resize, recompress, photograph, or reassemble arbitrary visual objects; replace labels and format blocks; splice regions from valid objects; replay old valid objects; mix versions and profiles; alter envelope bytes;

reconstruct capability-shaped objects; and copy or substitute any previously issued valid envelope. It may adaptively query a sealing oracle. For signature-forgery games it does not obtain an approved signing key unless compromise is explicitly modeled. Independence authorization does not rely on that restriction: an approved-key publisher may over-assert Distinct, but the assertion remains PublisherAsserted and cannot satisfy the high-authority provenance gate without external registry verification, attestation, or verifier establishment.

The games allow the adversary to choose any received object y . Their cryptographic conclusions therefore do not depend on a bound on the decoder's raw wrong-output rate. Correct canonical parsing, injectivity of DetEnc and PAE, and conformance to the transition relation define the modeled verifier.

We assume: (1) collision resistance for every digest algorithm admitted by policy; (2) existential signature unforgeability under adaptive chosen-message attack for approved, uncompromised keys; and (3) Assumption D for every profile on which the verifier reports agreement. If an approved key is exposed, an admitted digest is collision-broken, or L is nondeterministic under supposedly identical modeled inputs, the corresponding property is not claimed.

Let sealing oracle $O_{\text{seal}}(e)$ return a valid signature on $\mu(e)$ and record $\mu(e)$ in query set Q . Let $\text{VfyPipeline}_{\Pi}(y)$ be the staged verifier. A negligible function in security parameter λ is $\text{negl}(\lambda)$.

Properties 1–3 are deliberately **DSSE-class results for carrier transparency**, not new hardness results. They state that inserting a lossy decoder before a conventional signed envelope creates no authentication side channel.

6.2 Property 1: authenticated-envelope unforgeability

Theorem 2. Under signature unforgeability, an adversary with adaptive access to O_{seal} authenticates (e^*, σ^*) under the challenge key with $\mu(e^*) \notin Q$ only with probability

$$\Pr[G_{\text{auth}}^{\mathcal{A}} = 1] \leq \text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}) = \text{negl}(\lambda).$$

6.3 Property 2: atomic binding of payload kind and fields

Let $F(e)$ contain domain, object_version, codec_profile, payload_tag, and the complete selected payload. **Theorem 3.** If DetEnc and PAE are injective and canonical, then under signature unforgeability and collision resistance an adversary opens the same signed message to $F' \neq F(e^*)$, or substitutes a distinct referent under an admitted digest, only with probability

$$\Pr[G_{\text{bind}}^{\mathcal{A}} = 1] \leq \text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_1) + q_H \text{Adv}_H^{\text{cr}}(\mathcal{B}_2) = \text{negl}(\lambda),$$

where q_H bounds the relevant admitted digest queries.

6.4 Property 3: resistance to authenticated region mixing

Theorem 4. Under Theorems 2–3, an adversary that mixes arbitrary regions of valid rendered objects and arbitrary edits authenticates a new composite message $\mu(e^*) \notin Q$ only with probability

$$\Pr[G_{\text{mix}}^{\mathcal{A}} = 1] \leq \text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_1) + q_H \text{Adv}_H^{\text{cr}}(\mathcal{B}_2) = \text{negl}(\lambda).$$

Canonical re-encoding followed by signature verification reduces novel authenticated-envelope acceptance to EUF-CMA security. Alternative digest openings additionally reduce to collision resistance. Region mixing either yields such a new opening or decodes to an existing valid envelope, which is substitution or replay rather than forgery. The artifact preserves the detailed games and proof sketches.

6.5 Property 4: safe-promotion game

Game G_{promote} . The adversary chooses a received object and any promotion policy expressible by Section 4.2. After complete-set aggregation it may take any finite sequence of policy transitions. It wins if a policy transition changes the evidence-derived status, or if `execution_disagreed`, `not_reexecuted(reason)`, or `not_comparable(reason)` reaches basis `execution_agreement`; omitting a current report is verifier nonconformance, not a consumer strategy in the game.

By Theorem 1,

$$\Pr[G_{\text{promote}}^{\mathcal{A}} = 1] = 0.$$

This is an unconditional safety result for the specified authorization relation. The additional semantic statement attached to `report-level execution_agreed(J)` depends on L determinism and the correctness of the modeled execution classifier and supporting-evidence record.

6.6 Decoder outcomes and replay boundary

For intended envelope m , evaluation records three separate events:

If freshness is required, purpose, subject, nonce, issue time, and expiry are signed and checked against external state or policy. If freshness is intentionally out of scope, replay of an old valid object in the same context is not prevented. A newly signed stale execution claim is stale-3, not replay; re-execution detects it only when the committed observation differs from current deterministic execution.

7 Evaluation

7.1 Research questions

- **RQ1—binding soundness:** Does one signed typed object prevent accepted cross-profile substitution, IR/claim mix-and-match, payload-kind confusion, and bootstrap override?
- **RQ2—safe promotion:** Can an authenticated artifact lacking execution agreement acquire execution-agreement authority?
- **RQ3—incremental value of claims and re-execution:** Compared with A0 and A1, which drift, divergence, stale-3, and laundering failures become detectable, and how does the pattern depend on independence?

7.2 Completed formal evaluation

RQ1 is answered by the binding and mixing reductions under their explicit assumptions. RQ2 is addressed by Theorem 1 and supported by the finite-model safety check. The exhaustive claim covers all 65,536 quaternary relation vectors, all 32 per-axis relation/provenance/reference-presence cases, the exact 256-class verified projection, 1,024 conjunctive policies, all typed non-execution and incomparability reason sets, and complete singleton, pair, and triple report joins. It is not an exhaustive enumeration of disjunctive or non-monotone predicates; those are covered only by the structural argument that predicate shape over aggregate J cannot alter status-preserving edge provenance. The three negative controls exercise status relabeling, a corrupted publisher-asserted projection, and a policy that illegally inspects raw P . The artifact does not check stage monotonicity, signed-value authority, parser memory safety, cryptographic implementation correctness, arbitrary application policy code, or implementation-to-model correspondence. An anonymous artifact at <https://anonymous.4open.science/r/sealed2026-artifact> provides a single `verify_all.sh` entry point reproducing the finite-model checks, mutations, RQ3 matrix, and determinism outputs.

7.3 Executed RQ3 failure-injection design

RQ3 uses the real deterministic DSL L and four engine families. E1 is the native Rust reference interpreter. E2 is a self-hosted L runner with its own guest tokenizer, reader, normalizer, and machine. E3 is the native Rust product runner through the observation protocol. E4 is an engineered cross-language transliteration of E1 run on a distinct interpreter backend; a generated-native host for the same E4 transliteration is a fifth execution path, not a fifth engine lineage. E4 shares E1’s source and semantic-core lineage by construction even though its runtime backend and build toolchain differ. All are real repository implementations rather than simulations of engine separation.

The clean corpus contains only four programs exercising arithmetic, pairs/lists, non-tail recursion, and branching. This is a bounded mechanism surface, not a conformance suite or a program-diversity sample. It stays within the **84/205** primitive-row intersection advertised by the hosted engines. Here 205 is the full product capability registry and 84 is the subset supported by both hosted families; separately, **34/144** is the repositories’ historical count of all-family agreeing cases in a broader cross-family corpus, not a primitive-coverage fraction. RQ3 claims use the 84/205 intersection because every program and controlled primitive fault in this matrix stays within it, while 34/144 is retained as broader conformance context rather than converted into a coverage claim. Each program ran through E1, E2, E3, E4 on its cross-language interpreter host, and E4 on its generated-native host. `SignedProgramV1.canonical_ir.source` holds L’s source bytes and `normalized_ir` holds the real canonical program bytes; the verifier re-canonicalizes source before checking the pair. Raw observations are retained, but comparison uses a documented canonical language-level transcript projection.

RQ3 compares four verifier-policy scopes. A0 admits only the signed Program payload tag and has no execution claim; source is inside `canonical_ir` as specified above. A1 admits the signed

Table 1: Decoder and replay outcomes.

Event	Definition	Security interpretation
decoder wrong-output	$D_v(y) = m' \neq m$	carrier-quality failure; need not be zero for security
authenticated false accept	$D_v(y) = m' \neq m$ and origin/context checks accept	cryptographic false acceptance; target is observed zero plus cryptographic bound
valid substitution/replay	$D_v(y) = m'$, where m' is another or older valid envelope	not a forgery; controlled by signed context and freshness policy

Execution tag and the same final-result claim bytes used by B, but verifier re-execution is disabled, so applicable divergences remain `not_reexecuted(skipped_by_policy)`. B enables verifier re-execution over stdout and final values, with separately bound outcome and diagnostics. C enables re-execution over the full canonical transcript projection available across engines. The harness derives payload kind from the signed tag and commitment shape; A1 versus B is an externally fixed verifier policy, not an unsigned payload reinterpretation. No engine exposes a comparable language-level step trace, so C contains a canonical observable root/termination event trace plus an explicit `step_trace`: unavailable marker; C does not claim instruction- or reduction-level trace equivalence.

Each scope is evaluated under four recorded profiles. SHARED denotes a common lineage or fault domain, DISTINCT a separated domain, PARTIAL a mixed boundary, and UNKNOWN unavailable relation evidence. Only DISTINCT can satisfy independence. For this experiment the verifier records the displayed relations as `VerifierEstablished`; every axis therefore carries a required local derivation identifier, and `independence-profiles.json` retains the canonical derivation preimage and source/artifact digests named by that reference. These experiment-local annotations support attribution analysis; the RQ3 mechanism harness does not consume them in an authorization promotion decision, and they are neither registry verification nor third-party attestation:

The labels I0–I3 name evaluated profiles; they are not ranks. I2 and I3 separate complementary domains. In particular, E1→E4 is a distinct-runtime-backend and distinct-build-toolchain pairing, not a source-independent pairing. The transliteration inherits E1’s semantic decisions and can inherit defects in that logic. No evaluated profile combines independently authored source and semantic-core lineage with a distinct runtime backend. Partial is non-independent under promotion, exactly like Unknown; neither is silently rounded up to Distinct.

The ten conditions are a clean control plus stale-1, stale-2, stale-3, semantics-version drift, a single-operation backend fault, equal final results with divergent observable traces, normal/error outcome confusion, diagnostics promoted as success, and a common-mode Rust-runtime-backend fault. The matrix constructs them from six fixed fixture programs (mixed, past, semantics-drift, backend-fault, runtime-error, and shared-rust-lineage-fault), separately from the four-program clean feature corpus. All stale-3 injections commit an observation that differs from the current deterministic execution; the experiment therefore supports detection only for observation-divergent stale-3, not all stale claims. For the last condition (the retained harness class named `shared-Rust-lineage-fault`), fault-injected E1/E2/E3 artifacts introduce a one-input off-by-one

in the supported exact-addition host primitive; E4 is unmodified, and clean artifacts agree on the correct result. The mutation is therefore a Rust runtime-backend fault on the admitted hosted surface, not a change to the transliterated semantic logic or an observation-wrapper rewrite. Its detection by E4 tests a backend boundary only. This yields 4 scopes × 4 profiles × 10 conditions = 160 condition-combination cells. The 160 count is not a program-diversity sample size. A mechanical oracle fixes applicability and the expected detecting layer for every cell. Ground-truth injection class is recorded separately from verifier output. Fresh-process reruns test deterministic canonical and raw observations per path; cross-engine equality is required only for the canonical transcript projection.

8 Results

8.1 Scope and non-claims first

The primary RQ3 result is no longer toy-L or simulation-only: it executes real L programs through distinct artifacts and runtime arrangements, including the E1→E4 distinct-runtime-backend pairing. E4 is nevertheless a transliteration of E1, not an independently authored engine lineage. The result remains a **bounded real-L surface**, not whole-language or source-independent external validity. Hosted support is **84/205 primitive rows** and broader historical all-family agreement is **34/144 cases**; as Section 7.3 explains, the RQ3 claim denominator is 84/205 because the evaluated programs and injected primitive faults stay within that admitted common intersection, while 34/144 records a different, broader case corpus. The result establishes reproducible canonical agreement and controlled fault sensitivity on those programs and recorded fault domains only.

Only the **canonical transcript** is byte-identical across engines. Raw observations are not cross-engine byte-identical because engine identity, metrics, structured resource reports, and CLI-versus-product reporting differ. Raw observations are retained as evidence rather than erased. E2–E4 expose a structured product resource profile, while E1 exposes only a narrower recursion/process surface. All probes stayed within limits; equivalence under limit exhaustion was not proved. No comparable language-level step trace exists, so scope C is limited to the canonical observable root/termination event trace and explicitly does not establish step-by-step equivalence.

The E1→E4 comparison separates the runtime backend and build toolchain, while source, parser, and semantic-core lineage remain shared through transliteration. Every execution path also shares one operator, organization, physical machine, and language specification. Accordingly, the Section 4.1 evidence vector records verifier-established runtime/toolchain separation only: source-lineage, organizational, operator, hardware, and

Table 2: Recorded eight-axis RQ3 independence vectors.

Axis	I0: E3 → E3	I1: E3 → E1	I2: E3 → E2	I3: E1 → E4
source_lineage	SHARED	SHARED (common Rust lineage; distinct revision)	DISTINCT (guest L source)	SHARED (engineered transliteration)
parser_lineage	SHARED	SHARED (common lineage)	DISTINCT (guest tokenizer/reader)	SHARED (transliterated frontend)
semantics_core_lineage	SHARED	SHARED (common lineage)	DISTINCT (guest machine source)	SHARED (transliterated semantic choices)
build_toolchain	SHARED	SHARED (Cargo/rustc)	SHARED (Cargo wrapper)	DISTINCT (Rust/Cargo vs cross-language artifact)
runtime_backend	SHARED	SHARED (native Rust)	PARTIAL (guest L machine; shared Rust host)	DISTINCT (native Rust vs cross-language interpreter)
dependency_set	SHARED	PARTIAL (substantial overlap)	PARTIAL (shared host/contracts)	PARTIAL (separate engine trees; shared OS/contracts)
hardware_architecture	SHARED	SHARED	SHARED	SHARED
operator_or_organization	SHARED	SHARED	SHARED	SHARED

independent-specification replication are absent. Agreement remains case-specific compatibility evidence, not defect exclusion or an independent witness in every fault domain.

The construction accepts any EUF-CMA-secure signature scheme, with Ed25519 intended for deployment. The dependency-free mechanism harness uses HMAC-SHA256 only as a stand-in to demonstrate the single-authenticator construction over the same domain-separated bytes. The matrix evaluates commitment scope, re-execution, and typed promotion—not public-key implementation security. It supplies no execution proof, no historical evidence that the attributed engine ran, no verifier-derived fault classification, and no claim beyond the signed commitment scope and recorded independence vector.

8.2 RQ3: re-execution adds detection that signed IR alone lacks

The mechanical oracle passed **160/160 oracle cells**. A second fresh-process full run reproduced all six result artifacts byte-for-byte, including the 16 common-mode Rust-runtime-backend cells. Four real programs across five execution paths produced **byte-identical canonical transcripts**. Every path reproduced its own canonical and raw observation bytes exactly in that second fresh process, confirming canonical determinism for the tested corpus. Raw observations, as disclosed above, were reproducible per path but not byte-identical across engines.

Table cells name the detecting layer. PASS is a clean verification control. Under A0, N/A marks an execution-claim attack that the signed Program payload structurally cannot contain. Under A1, MISS marks an authenticated signed claim whose divergence remains undetected because re-execution is disabled. This RQ3 mechanism harness stops at `VerificationStatus`; it does not instantiate the verifier-store aggregation or action-promotion decision, which is evaluated separately by the finite model.

SIG, FRESH, and REEXEC are verifier-reported detecting layers: signature, nonce/freshness, and deterministic re-execution agreement. The fault names are controlled harness ground truth, not diagnoses emitted by the verifier.

The load-bearing comparison is stale-3. Section 3.3 defines it as an old result signed as a new execution claim. A0 has no claim field

or consumer action for that result, so stale-3 is structurally unconstructible and correctly N/A. A1 carries the stale `SignedExecutionEnvelopeV1` and authenticates its atomic binding, but re-execution is disabled, so every A1 stale-3 cell is a formally valid MISS. B and C caught all injected stale-3 cells at **I0–I3** because every committed state observation differed from current deterministic execution. This does not generalize to an observation-equal stale claim. The A1-to-B/C comparison therefore isolates verifier-side re-execution value without redefining the attack, while A0-to-A1 isolates addition and signed binding of the claim.

Independence changes which engine faults are visible only when re-execution occurs. A0 has no claim and those cells are N/A; A1 carries each divergent claim but misses it throughout. Under B and C, the controlled single-operation backend fault was missed by I0, whose publisher and verifier reran the same affected artifact, and caught by **I1–I3**. Semantics-version drift had the same profile. The added engine-side common-mode fault further separates the profiles: E1/E2/E3 share an activated off-by-one in a Rust host primitive, so B/C at I1 and I2 agree on the same wrong result and miss it, while E4’s unmodified cross-language backend disagrees and detects it in I3. This demonstrates measured value from runtime-backend independence on one controlled cell. It does **not** demonstrate source-lineage independence: E4 is an engineered transliteration, so a defect in the shared semantic choices or transliterated evaluator logic could be inherited and missed. The cell also does not show that the injected defect occurs naturally or that agreement excludes common defects.

Commitment scope is also material. B cannot observe an equal-result divergence confined to the available canonical event trace; C caught that mutation at I0–I3. Because C is not a language-level step trace, the claim is specifically about the committed observable root/termination events, not intermediate reduction equivalence.

8.3 Controlled prior and external-validity agenda

The prior toy-L experiment remains a controlled mechanism check: its 81/81 oracle cells and canonical step tracing isolate commitment-scope behavior, but it is not the paper’s primary evidence. The real-L matrix supersedes it for external validity.

Table 3: RQ3 detecting layer and harness ground-truth injection class.

Injection	A0/I0	A0/I1	A0/I2	A0/I3	A1/I0	A1/I1	A1/I2	A1/I3	B/I0	B/I1	B/I2	B/I3	C/I0	C/I1	C/I2	C/I3
clean	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
stale-1	N/A	N/A	N/A	N/A	SIG	SIG	SIG	SIG	SIG	SIG	SIG	SIG	SIG	SIG	SIG	SIG
stale-2	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH	FRESH
stale-3	N/A	N/A	N/A	N/A	MISS	MISS	MISS	MISS	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC
semantics-version-drift	N/A	N/A	N/A	N/A	MISS	MISS	MISS	MISS	MISS	REEXEC	REEXEC	REEXEC	MISS	REEXEC	REEXEC	REEXEC
single-op backend fault	N/A	N/A	N/A	N/A	MISS	MISS	MISS	MISS	MISS	REEXEC	REEXEC	REEXEC	MISS	REEXEC	REEXEC	REEXEC
equal-result trace divergence	N/A	N/A	N/A	N/A	MISS	MISS	MISS	MISS	MISS	MISS	MISS	MISS	REEXEC	REEXEC	REEXEC	REEXEC
outcome confusion	N/A	N/A	N/A	N/A	MISS	MISS	MISS	MISS	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC
diagnostics as success	N/A	N/A	N/A	N/A	MISS	MISS	MISS	MISS	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC	REEXEC
Rust-backend common-mode	N/A	N/A	N/A	N/A	MISS	MISS	MISS	MISS	MISS	MISS	MISS	REEXEC	MISS	MISS	MISS	REEXEC

Under scopes B/C, its single-operation backend fault is missed at I0 and caught at I1–I3 because each later pairing separates that activated implementation fault; A0 has no claim and A1 does not re-execute. The added real-L Rust-runtime-backend cell further shows B/C misses at I1/I2 and detection by E4’s distinct cross-language backend in I3.

Future work follows directly from the remaining bounds: replace the four-program mechanism corpus with a systematic conformance suite and expand beyond the current **84/205** hosted primitive-row surface and **34/144** broader historical all-family case agreement toward whole-language and whole-corpus convergence; compare limit-exhaustion outcomes and richer I/O contracts; add a genuinely comparable language-level step trace if the engines expose one; obtain true source-lineage-independent replication through an independently authored implementation rather than a transliteration; replicate with independently maintained engines operated by different organizations and people on different machines; and repeat the carrier evaluation through independent physical print/camera channels rather than a single-operator simulated channel. These are extensions of external validity, not facts established here.

9 Threat-Model Boundaries and Non-Claims

The following boundaries are part of the result, not deferred implementation caveats:

- **Execution assurance.** The envelope supports an attributed claim and verifier agreement, not proof that the engine ran, an execution or historical proof such as a SNARK, STARK, or TEE attestation, or defect exclusion. Agreement only adds differential sensitivity. The games do not estimate attack prevalence.
- **Independence.** I3 separates runtime and toolchain but shares transliterated source, parser, and semantic core. Every path shares operator, organization, machine, and specification. It is not source, organization, operator, hardware, specification, or publisher-environment replication. Strength follows the actual vector, and I0–I3 are not ranks.
- **Semantic scope.** Claims cover canonical L IR, not source-language equivalence. Four programs and six fixtures occupy **84/205** hosted rows. The 160 cells are conditions, and **34/144** is only broader historical agreement. This is no

conformance, diversity, whole-language, arbitrary-native-code, or whole-program result. Canonical equality is not raw equality or completeness for omitted axes. Exhaustion equivalence is unproved, and scope C has no language-level step trace.

- **Trust infrastructure.** An embedded key is not a trust root. External trust, profile, semantics, decoder, and verifier policies are assumed correct. We do not solve policy correctness, key custody, or revocation. Replay depends on external enforcement of signed freshness. The system supplies no log or trusted timestamp. Offline means no online manifest or sidecar, not no signature infrastructure.
- **Carrier.** Robust mode authenticates recovered envelope bytes, not received pixels.

10 Related Work

Exact-byte provenance and supply chains. DSSE authenticates serialized payload bytes, in-toto binds supply-chain steps and artifacts, SLSA standardizes provenance, and Sigstore makes software-signing identities and events auditable [6, 9, 10, 12]. We use that evidence substrate for a downstream typed authorization machine and do not replace its trust, identity, revocation, or transparency functions.

Information flow, tpestate, and attestation. Information-flow lattices constrain flows between labels, tpestate permits operations only from established states, and RATS separates attestation evidence and appraisal from a relying party’s authorization decision [1, 3, 4, 11]. Our delta is an immutable execution-evidence status paired with an explicit authorization basis, bound to deterministic comparison and evaluated on real engines.

Proof carrying and translation validation. Proof-carrying code attaches a machine-checkable safety proof, which our execution-claim-carrying envelope does not provide [5]. Translation validation checks a particular output against source semantics, while our case-specific comparison checks a verifier observation against a signed engine-attributed observation and does not validate any source-to-L translation [7].

Carrier-adjacent work. C2PA separates cryptographic hard binding from soft fingerprints or watermarks, while QRscript shows that executable bytecode can be carried by standard visual symbols [2, 8]. We require exact envelope recovery followed by signature verification and make no carrier novelty claim.

11 Discussion

11.1 What the execution commitment adds

A signed expected result is an attributed comparison point, not proof or fault diagnosis. A0 carries only the program. A1 authenticates the publisher’s claim but establishes no agreement. B/C detected every observation-divergent stale-3 claim. Under B/C, the controlled backend fault and semantics drift were missed at same-artifact I0 and exposed at I1–I3, while the Rust-runtime common-mode fault was missed at I1/I2 and exposed only by E4 at I3. E1→E4 separates runtime backend and build toolchain but shares transliterated source, parser, and semantic-core lineage. Agreement therefore shows case-specific compatibility, not historical execution, diagnosis, source-independent replication, or defect exclusion.

11.2 Policy usability

The explicit origin-only path avoids undocumented bypasses for unsupported or resource-limited artifacts. `not_reexecuted` preserves its four reasons, and `not_comparable(reason)` has the same typed option after an impossible comparison. Neither becomes execution agreement. Complete current-report aggregation prevents skipping or cherry-picking from defeating a later disagreement.

11.3 Channel and carrier independence

We evaluated a representative lossy visual carrier, rejected a custom design against a strong standard mosaic, and therefore use standard symbols with sharding and outer erasure correction. Full definitions and results are in the artifact.

11.4 From model to implementation

The artifact exhausts the finite join, reason, projection, status, basis, scope-pin, and conjunctive-policy domains and catches three mutations. It neither enumerates arbitrary predicates nor proves implementation conformance. Structurally, any projected predicate still guards only the grant from $A_I(J)$ and cannot relabel status. Implementations must centralize complete-set aggregation, preserve immutable typed status and reasons, require references for verified provenance, pin verifier scope, and test canonical encoding, parser differences, signature misuse, resource exhaustion, report ordering, and the mutations.

12 Conclusion

We make the separation of authenticated origin from authority explicit with split signed types and a typed `VerificationStatus × AuthorizationBasis` over complete report aggregates. Policy cannot rewrite evidence, and disagreement supersedes fallback. High-authority independence requires referenced, verified-provenance Distinct axes, so a signer cannot promote its own assertion. The finite model catches three laundering mutations but is safety evidence for the abstraction, not an implementation-conformance proof.

The real-L matrix passed **160/160 condition cells**, reproduced all artifacts byte-for-byte, and produced **byte-identical canonical transcripts** for four programs over five paths. Observation-divergent stale-3 was N/A at A0, missed at A1, and detected

throughout B/C. Under B/C the backend fault was missed at I0 and caught at I1–I3. The Rust-runtime common-mode fault was missed at I1/I2 and caught by E4 at I3. Because E1→E4 shares transliterated source and semantic-core lineage, that cell measures backend separation only.

This bounded corpus has four clean programs, six fixtures, **84/205** hosted primitive rows, and **34/144** broader historical agreeing cases. The 160 cells are conditions, not programs. Raw observations differ, exhaustion equivalence is unproved, scope C lacks a comparable language-level step trace, and all paths share one operator, organization, machine, and specification. Conformance-scale coverage, a non-transliterated implementation, independent operation, and physical-channel replication remain future work. No whole-language or source-independent result is claimed.

References

- [1] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. 2023. *Remote ATestation procedureS (RATS) Architecture*. Technical Report RFC 9334. Internet Engineering Task Force. doi:10.17487/RFC9334
- [2] Coalition for Content Provenance and Authenticity. [n.d.]. C2PA Technical Specification 2.4: Binding to Content. <https://spec.c2pa.org/specifications/specifications/2.4/specs/ContentCredentials.html>.
- [3] Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236–243. doi:10.1145/360051.360056
- [4] Andrew C. Myers and Barbara Liskov. 1997. A Decentralized Model for Information Flow Control. In *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*. 129–142. doi:10.1145/268998.266669
- [5] George C. Necula. 1997. Proof-Carrying Code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 106–119. doi:10.1145/263699.263712
- [6] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. 2022. Sigstore: Software Signing for Everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2353–2367. doi:10.1145/3548606.3560596
- [7] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 1384)*. 151–166.
- [8] Stefano Scanzio, Gianluca Cena, and Adriano Valenzano. 2024. QRscript: Embedding a Programming Language in QR Codes to Support Decision and Management. arXiv:2404.05073 [cs.PL] <https://arxiv.org/abs/2404.05073>
- [9] Secure Systems Lab. 2024. Dead Simple Signing Envelope (DSSE), Protocol v1.0.2. <https://github.com/secure-systems-lab/dsse/blob/master/protocol.md>.
- [10] SLSA Community. [n.d.]. SLSA v1.2: Provenance. <https://slsa.dev/spec/v1.2/provenance>.
- [11] Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171. doi:10.1109/TSE.1986.6312929
- [12] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium (USENIX Security 19)*. 1393–1410.